# 1. Tên: MVC (Model - View - Controller)

Design Pattern được áp dụng trong dự án này là MVC (Model-View-Controller). Đây là mẫu kiến trúc phân tách ứng dụng thành ba thành phần độc lập:

- Model (SQLConnection): Quản lý dữ liệu và logic kết nối đến hệ quản trị CSDL.
- View (IoTView): Quản lý giao diện hiển thị dữ liệu trên các nền tảng hệ điều hành khác nhau.
- Controller (IoTController): Điều phối giữa Model và View theo yêu cầu của người dùng.

# 2. Vấn đề (Problem)

Nhóm thiết kế IoTLab cần xây dựng một thư viện phần mềm theo mô hình MVC để đáp ứng các yêu cầu sau:

**Vấn đề 1 — Kết nối CSDL đa loại:** Hệ thống cần kết nối được với nhiều hệ quản trị CSDL khác nhau (MySQL, PostgreSQL, SQLite) và có thể mở rộng thêm loại CSDL mới mà không cần thay đổi cấu trúc hiện tại.

**Vấn đề 2 — Hiển thị đa nền tảng:** Giao diện hiển thị phải hoạt động được trên nhiều hệ điều hành (Windows, iOS, MacOS), mỗi hệ điều hành có cách cài đặt giao diện riêng.

**Vấn đề 3 — Tách biệt logic:** Logic kết nối CSDL và logic hiển thị phải hoàn toàn độc lập, dễ thay thế và kiểm thử riêng lẻ mà không ảnh hưởng đến các thành phần còn lại.

# 3. Giải pháp (Solution)

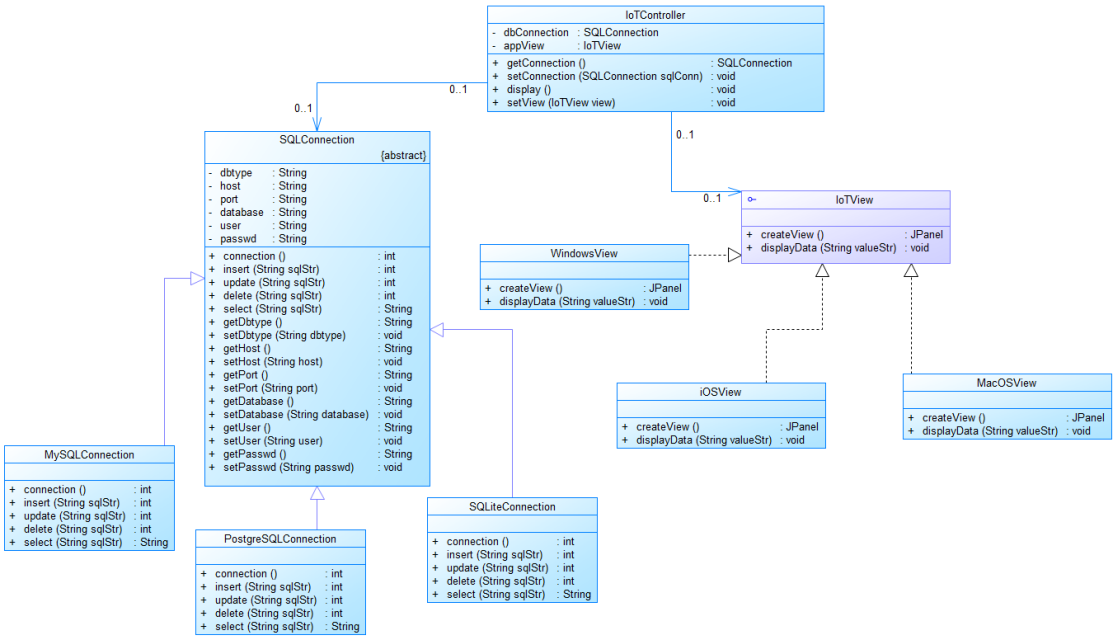
Giải pháp áp dụng mô hình **MVC** kết hợp với **Abstract Class** và **Interface** để tối ưu hóa khả năng mở rộng:

- **SQLConnection** được định nghĩa là một **lớp trừu tượng (Abstract Class)** chứa các thuộc tính dùng chung, với các lớp con cài đặt cụ thể cho từng hệ quản trị: **MySQLConnection**, **PostgreSQLConnection**, **SQLiteConnection**.

- **IoTView** được định nghĩa là một **giao diện (Interface)** quy định các phương thức hiển thị, với các lớp cài đặt cụ thể cho từng hệ điều hành: **WindowsView**, **iOSView**, **MacOSView**.

- **IoTController** đóng vai trò điều phối, giữ tham chiếu đến đối tượng trừu tượng của **SQLConnection** và **IoTView** để thực hiện kết nối dữ liệu và hiển thị tương ứng với yêu cầu người dùng

## 3.1. Sơ đồ lớp (Class Diagram)




Mô tả cấu trúc sơ đồ lớp:

- SQLConnection (abstract) là lớp cha, có các lớp con kế thừa là MySQLConnection, PostgreSQLConnection, SQLiteConnection.

- IoTView là một Interface (Giao diện) được thực thi (implement) bởi các lớp WindowsView, iOSView, MacOSView.

- IoTController có quan hệ liên kết (association) với SQLConnection và IoTView (thể hiện đúng việc Controller chứa 2 đối tượng này làm thuộc tính dbConnection và appView ).

## 3.2. Mô tả các thành phần (Class Descriptions)

Phần này mô tả chi tiết từng lớp, bao gồm mục đích, thuộc tính và phương thức.

### 3.2.1. Lớp: SQLConnection (Abstract — Tầng Model)

**Mục đích:** Lớp trừu tượng định nghĩa giao diện thống nhất cho tất cả các đối tượng kết nối CSDL. Cung cấp các phương thức get/set chung (không phụ thuộc DBMS) và 5 phương thức trừu tượng phụ thuộc DBMS mà các lớp con phải cài đặt.
**Ràng buộc:** Không thể khởi tạo trực tiếp. Các lớp con bắt buộc phải override 5 phương thức trừu tượng: connection(), insert(), update(), delete(), select().
**Persistent:** Không (không lưu trạng thái, tạo và cấu hình khi chạy chương trình).

**<u>Attribute Descriptions</u>**

1. **Attribute:** *dbtype*
   **Type:** String
   **Description:** Loại Database Management System. Ví dụ: "mysql", "postgresql", "sqlite". Không phân biệt chữ hoa/thường (ignore case sensitive).
   **Constraints:** Giá trị phải là một trong: "mysql", "postgresql", "sqlite".
2. **Attribute:** *host*
   **Type:** String
   **Description:** Địa chỉ IP hoặc hostname của database server. Ví dụ: "192.168.1.100" hoặc "localhost".
   **Constraints:** Không được để trống. Bỏ qua khi dbtype = "sqlite".
3. **Attribute:** *port*
   **Type:** String
   **Description:** Số cổng kết nối, biểu diễn bằng chuỗi số. Ví dụ: "3306" (MySQL), "5432" (PostgreSQL).
   **Constraints:** Phải là chuỗi số nguyên dương. Bị bỏ qua khi dbtype = "sqlite".
4. **Attribute:** *database*

**Type:** String
**Description:** Tên của CSDL cần kết nối. Với SQLite, đây là đường dẫn đến file .db. Ví dụ: "iot_db" hoặc "D:/CT240/iot.db".
**Constraints:** Không được để trống.

5. **Attribute:** *user*
   **Type:** String
   **Description:** Tên đăng nhập (username) để xác thực với CSDL.
   **Constraints:** Bỏ qua khi dbtype = "sqlite" (SQLite không yêu cầu xác thực).
6. **Attribute:** *passwd*
   **Type:** String
   **Description:** Mật khẩu (password) để đăng nhập vào CSDL.
   **Constraints:** Bỏ qua khi dbtype = "sqlite".

**<u>Method Descriptions</u>**

*Getter và Setter — chung cho tất cả DBMS (không phụ thuộc DBMS):*

7. **Method:** getDbtype() / setDbtype(String v)
   **Return Type:** String / void
   **Parameters:** v — giá trị chuỗi cần gán cho dbtype (chỉ cho setter)
   **Return value:** getDbtype() trả về giá trị dbtype hiện tại; setDbtype() không trả về giá trị.
   **Processing logic:** get: trả về giá trị thuộc tính. set: gán giá trị tham số vào thuộc tính tương ứng.

→ Tương tự cho các cặp: getHost/setHost, getPort/setPort, getDatabase/setDatabase, getUser/setUser, getPasswd/setPasswd.

*Phương thức trừu tượng — phụ thuộc DBMS (các lớp con bắt buộc phải override):*

8. **Method:** connection()
   **Return Type:** int
   **Parameters:** Không có
   **Return value:** 0 nếu kết nối thành công; mã lỗi khi thất bại
   **Pre-condition:** Các thuộc tính (host, port, database, user, passwd) phải được thiết lập trước khi gọi phương thức
   **Post-condition:** Kết nối đến DBMS được mở thành công
   **Processing logic:** Gọi driver DBMS tương ứng để mở kết nối dựa trên các thuộc tính đã lưu. Mỗi lớp con tự cài đặt riêng.
9. **Method:** insert(String sqlStr)
   **Return Type:** int
   **Parameters:** sqlStr — chuỗi lệnh SQL INSERT cần thực thi **Return value:** 0 nếu thành công; mã lỗi thuộc [1..99] nếu thất bại **Pre-condition:** Kết nối CSDL phải được mở bằng connection() **Post-condition:** Một bản ghi mới được thêm vào CSDL
   **Processing logic:** Thực thi chuỗi lệnh SQL INSERT qua driver DBMS. Trả về 0 nếu OK, [1..99] nếu lỗi.
10. **Method:** update(String sqlStr)
    **Return Type:** int
    **Parameters:** sqlStr — chuỗi lệnh SQL UPDATE cần thực thi **Return value:** 0 nếu thành công; mã lỗi thuộc [1..99] nếu thất bại **Pre-condition:** Kết nối CSDL phải được mở bằng connection() **Post-condition:** Bản ghi được cập nhật trong CSDL
    **Processing logic:** Thực thi chuỗi lệnh SQL UPDATE qua driver DBMS. Trả về 0 nếu OK, [1..99] nếu lỗi.
11. **Method:** delete(String sqlStr)
    **Return Type:** int
    **Parameters:** sqlStr — chuỗi lệnh SQL DELETE cần thực thi **Return value:** 0 nếu thành công; mã lỗi thuộc [1..99] nếu thất bại **Pre-condition:** Kết nối CSDL phải được mở bằng connection() **Post-condition:** Bản ghi chỉ định bị xóa khỏi CSDL
    **Processing logic:** Thực thi chuỗi lệnh SQL DELETE qua driver DBMS. Trả về 0 nếu OK, [1..99] nếu lỗi.
12. **Method:** select(String sqlStr)
    **Return Type:** String
    **Parameters:** sqlStr — chuỗi lệnh SQL SELECT cần thực thi
    **Return value:** Chuỗi JSON chứa kết quả truy vấn; "[]" nếu không có dữ liệu
    **Pre-condition:** Kết nối CSDL phải được mở bằng connection()
    **Post-condition:** Không thay đổi dữ liệu; kết quả được trả về dạng JSON **Processing logic:** Thực thi lệnh SELECT và chuyển kết quả ResultSet sang chuỗi JSON để trả về.

### 3.2.2. Lớp: MySQLConnection (Cụ thể)

**Mục đích:** Lớp con cụ thể của SQLConnection, cài đặt tất cả 5 phương thức trừu tượng bằng driver MySQL (JDBC). Sử dụng khi dbtype = "mysql".
**Ràng buộc:** Cổng mặc định là 3306 nếu không được chỉ định.

Kế thừa toàn bộ thuộc tính và getter/setter từ SQLConnection. Override 5 phương thức trừu tượng:

- connection() — dùng chuỗi kết nối JDBC MySQL: jdbc:mysql://host:port/database

- insert(), update(), delete() — thực thi lệnh SQL qua MySQL PreparedStatement, trả về 0 nếu thành công hoặc mã lỗi [1..99]
- select() — thực thi lệnh SELECT, chuyển ResultSet thành chuỗi JSON trả về

### 3.2.3. Lớp: PostgreSQLConnection (Cụ thể)

**Mục đích:** Lớp con cụ thể của SQLConnection cho PostgreSQL. Sử dụng khi dbtype = "postgresql".
**Ràng buộc:** Cổng mặc định là 5432 nếu không được chỉ định.

Kế thừa toàn bộ thuộc tính và getter/setter từ SQLConnection. Override 5 phương thức trừu tượng với chuỗi kết nối: jdbc:postgresql://host:port/database.

### 3.2.4. Lớp: SQLiteConnection (Cụ thể)

**Mục đích:** Lớp con cụ thể của SQLConnection cho SQLite. Sử dụng khi dbtype = "sqlite".
**Ràng buộc:** Thuộc tính port BỊ BỎ QUA. Thuộc tính database chứa đường dẫn đến file .db.

Kế thừa toàn bộ thuộc tính và getter/setter từ SQLConnection. Override 5 phương thức trừu tượng với chuỗi kết nối: jdbc:sqlite:<đường-dẫn-file>.

### 3.2.5. Lớp: IoTView (Interface— Tầng View)

**Mục đích:** Giao diện định nghĩa khung hiển thị thống nhất cho tất cả các đối tượng dữ liệu. Cung cấp 2 phương thức mẫu phụ thuộc vào nền tảng hệ điều hành. Các lớp thực thi bắt buộc phải cài đặt cả hai phương thức này.
**Ràng buộc:** Không thể khởi tạo trực tiếp. Phương thức displayData() nên được gọi sau khi createView() đã chạy.

**<u>Method Descriptions</u>**

13. **Method:** createView()
    **Return Type:** JPanel
    **Parameters:** Không có
    **Return value:** Đối tượng JPanel đại diện cho cửa sổ hiển thị
    **Pre-condition:** Không có
    **Post-condition:** Panel giao diện được khởi tạo phù hợp với nền tảng hiện tại **Processing logic:** Mỗi lớp con tạo JPanel bằng cách sử dụng bộ công cụ giao diện của hệ điều hành tương ứng.
14. **Method:** displayData(String valueStr)
    **Return Type:** void
    **Parameters:** valueStr — chuỗi JSON chứa dữ liệu cần hiển thị
    **Return value:** Không có
    **Pre-condition:** createView() phải được gọi trước; valueStr phải là JSON hợp lệ
    **Post-condition:** Dữ liệu từ valueStr được hiển thị lên giao diện
    **Processing logic:** Phân tích chuỗi JSON và vẽ dữ liệu lên panel của nền tảng hiện tại.

### 3.2.6. Lớp: WindowsView (Cụ thể)

**Mục đích:** Lớp con cụ thể của IoTView cho nền tảng Windows. Cài đặt createView() và displayData() sử dụng các thành phần giao diện của Windows.

### 3.2.7. Lớp: iOSView (Cụ thể)

**Mục đích:** Lớp con cụ thể của IoTView cho nền tảng iOS. Cài đặt createView() và displayData() sử dụng các thành phần giao diện của iOS.

### 3.2.8. Lớp: MacOSView (Cụ thể)

**Mục đích:** Lớp con cụ thể của IoTView cho nền tảng MacOS. Cài đặt createView() và displayData() sử dụng các thành phần giao diện của MacOS.

### 3.2.9. Lớp: IoTController (Tầng Controller)

**Mục đích:** Lớp điều khiển trong mô hình MVC. Giữ tham chiếu đến một SQLConnection và một IoTView, điều phối quá trình lấy dữ liệu từ CSDL và hiển thị lên giao diện.
**Ràng buộc:** Thuộc tính dbConnection và appView phải được thiết lập trước khi gọi display().

**Attribute Descriptions**

15. **Attribute:** *dbConnection*
    **Type:** SQLConnection
    **Description:** Đối tượng kết nối CSDL hiện tại (bất kỳ lớp con cụ thể nào của SQLConnection).
    **Constraints:** Phải được gán trước khi gọi display().
16. **Attribute:** *appView*
    **Type:** IoTView
    **Description:** Đối tượng hiển thị hiện tại (bất kỳ lớp con cụ thể nào của IoTView).
    **Constraints:** Phải được gán trước khi gọi display().

**Method Descriptions**

17. **Method:** getConnection()
    **Return Type:** SQLConnection
    **Parameters:** Không có
    **Return value:** Đối tượng SQLConnection hiện tại của controller
    **Pre-condition:** dbConnection đã được thiết lập
    **Processing logic:** Trả về thuộc tính dbConnection.
18. **Method:** setConnection(SQLConnection sqlConn)
    **Return Type:** void
    **Parameters:** sqlConn — đối tượng SQLConnection cần gán
    **Return value:** Không có
    **Post-condition:** dbConnection được gán bằng sqlConn
    **Processing logic:** Gán sqlConn vào thuộc tính dbConnection.
19. **Method:** display() **Return**
    **Type:** void **Parameters:**
    Không có **Return value:**
    Không có
    **Pre-condition:** dbConnection đã kết nối; appView đã được thiết lập **Post-condition:** Dữ liệu được lấy từ CSDL và hiển thị lên giao diện **Phương thức gọi:** dbConnection.connection(), dbConnection.select(), appView.createView(), appView.displayData()
    **Processing logic:** Gọi connection() để mở kết nối, rồi gọi select() để lấy dữ liệu JSON, sau đó gọi createView() và displayData() để hiển thị.

## 4. Kết quả (Consequence)

Thiết kế MVC với các lớp trừu tượng mang lại những ưu điểm và đánh đổi sau:

**Ưu điểm:**

- Mở rộng dễ dàng: Có thể thêm DBMS mới (Oracle, MongoDB,...) bằng cách tạo lớp con SQLConnection mới mà không sửa code hiện có (tuân theo nguyên tắc Mở/Đóng).
- Hỗ trợ đa nền tảng: Hỗ trợ nền tảng mới (Linux, Android,...) bằng cách tạo lớp con IoTView mới.
- Tách biệt mối quan tâm: Controller không phụ thuộc vào cài đặt cụ thể, chỉ phụ thuộc vào giao diện trừu tượng, giúp dễ thay thế khi chạy.
- Dễ kiểm thử: Mỗi thành phần (Model, View, Controller) có thể kiểm thử độc lập bằng mock object.

**Hạn chế:**

- Số lượng lớp tăng: Số lớp tăng tuyến tính theo số loại DBMS và nền tảng hỗ trợ.
- Phức tạp hơn: Với dự án nhỏ, tầng trừu tượng có thể gây thêm độ phức tạp không cần thiết.

## 5. Minh họa code (Java)

Đoạn code Java sau minh họa khai báo SQLConnection, IoTView, IoTController và các lớp cài đặt cụ thể ủa chúng

File code nè:

```
// ============================================================
// 1. FILE: SQLConnection.java (Model - Abstract Class)
// ============================================================
public abstract class SQLConnection {
    private String dbtype;
    private String host;
    private String port;
    private String database;
    private String user;
    private String passwd;

    // Các phương thức Get/Set không phụ thuộc DBMS [cite: 14]
    public String getDbtype() { return dbtype; }
    public void setDbtype(String dbtype) { this.dbtype = dbtype; }
    public String getHost() { return host; }
    public void setHost(String host) { this.host = host; }
    public String getPort() { return port; }
    public void setPort(String port) { this.port = port; }
    public String getDatabase() { return database; }
    public void setDatabase(String database) { this.database = database;
}
    public String getUser() { return user; }
    public void setUser(String user) { this.user = user; }
    public String getPasswd() { return passwd; }
    public void setPasswd(String passwd) { this.passwd = passwd; }

    // Các phương thức trừu tượng phụ thuộc DBMS [cite: 15]
    public abstract int connection();
    public abstract int insert(String sqlStr);
    public abstract int update(String sqlStr);
    public abstract int delete(String sqlStr);
    public abstract String select(String sqlStr);
}

// ============================================================
// 2. FILE: MySQLConnection.java
// ============================================================
```

```
public class MySQLConnection extends SQLConnection {
    @Override
    public int connection() { return 0; }
    @Override
    public int insert(String sqlStr) { return 0; }
    @Override
    public int update(String sqlStr) { return 0; }
    @Override
    public int delete(String sqlStr) { return 0; }
    @Override
    public String select(String sqlStr) {
        return "{\"database\": \"MySQL\", \"data\": \"Dữ liệu từ
MySQL\"}";
    }
}

// ============================================================
// 3. FILE: PostgreSQLConnection.java
// ============================================================
public class PostgreSQLConnection extends SQLConnection {
    @Override
    public int connection() { return 0; }
    @Override
    public int insert(String sqlStr) { return 0; }
    @Override
    public int update(String sqlStr) { return 0; }
    @Override
    public int delete(String sqlStr) { return 0; }
    @Override
    public String select(String sqlStr) {
        return "{\"database\": \"PostgreSQL\", \"data\": \"Dữ liệu từ
PostgreSQL\"}";
    }
}

// ============================================================
// 4. FILE: SQLiteConnection.java
// ============================================================
public class SQLiteConnection extends SQLConnection {
    @Override
    public int connection() { return 0; }
    @Override
    public int insert(String sqlStr) { return 0; }
    @Override
    public int update(String sqlStr) { return 0; }
    @Override
    public int delete(String sqlStr) { return 0; }
    @Override
    public String select(String sqlStr) {
        return "{\"database\": \"SQLite\", \"data\": \"Dữ liệu từ
SQLite\"}";
    }
}

// ============================================================
// 5. FILE: IoTView.java (View - Interface)
// ============================================================
import javax.swing.JPanel;

public interface IoTView {
    public JPanel createView(); // Phương thức tạo cửa sổ [cite: 34]
    public void displayData(String valueStr); // Phương thức hiển thị
JSON [cite: 35]
}

// ============================================================
// 6. FILE: WindowsView.java
```

```
// ============================================================
import javax.swing.JPanel;

public class WindowsView implements IoTView {
    @Override
    public JPanel createView() { return new JPanel(); }
    @Override
    public void displayData(String valueStr) {
        System.out.println("Windows Display (JSON): " + valueStr);
    }
}

// ============================================================
// 7. FILE: iOSView.java
// ============================================================
import javax.swing.JPanel;

public class iOSView implements IoTView {
    @Override
    public JPanel createView() { return new JPanel(); }
    @Override
    public void displayData(String valueStr) {
        System.out.println("iOS Display (JSON): " + valueStr);
    }
}

// ============================================================
// 8. FILE: MacOSView.java
// ============================================================
import javax.swing.JPanel;

public class MacOSView implements IoTView {
    @Override
    public JPanel createView() { return new JPanel(); }
    @Override
    public void displayData(String valueStr) {
        System.out.println("MacOS Display (JSON): " + valueStr);
    }
}

// ============================================================
// 9. FILE: IoTController.java (Controller)
// ============================================================
public class IoTController {
    private SQLConnection dbConnection;
    private IoTView appView;

    public SQLConnection getConnection() { return this.dbConnection; } //
[cite: 42]

    public void setConnection(SQLConnection sqlConn) { // [cite: 44]
        this.dbConnection = sqlConn;
    }

    public void setAppView(IoTView appView) {
        this.appView = appView;
    }

    public void display() { // [cite: 45]
        if (dbConnection != null && appView != null) {
            String data = dbConnection.select("SELECT * FROM IoTData");
            appView.displayData(data);
        }
    }
}
```